

3장

축소 정복 기법

DECREASE AND CONQUER

정답을 찾기 위해 모든 곳을 다 뒤져야만 할까?

학습 목표

핵심 질문 “정답을 찾기 위해 모든 곳을 다 뒤져야만 할까?”

학습 포인트

축소 정복 기법의 기본 원리

문제를 같은 유형의 더 작은 문제로 바꾸어 반복적으로 해결하는 전략을 이해한다.

완전 탐색의 한계 극복

입력이 커질 때 폭발하는 계산량을 어떻게 줄일 수 있는지 학습한다.

문제 크기를 줄이는 3가지 전략

동일 크기 축소 · 동일 비율 축소 · 가변 크기 축소를 구분해 파악한다.

이번 장의 흐름

3.1 축소 정복이란 무엇인가

3.2 축소의 세 가지 유형

3.3 구현: 재귀와 반복

3.4 탐색 알고리즘

3.5 유클리드와 거듭제곱

3.6 정렬과 선택 문제

3.7 Coding Test

Q & A

정의에서 출발해 세 가지 축소 유형을 구분하고, 재귀·반복 구현을 거쳐 탐색·거듭제곱·정렬·선택 문제로 확장합니다.

거대한 문제를 마주했을 때

완전 탐색은 모든 경우를 확인합니다. 이제는 문제의 크기를 줄여 봅시다.



순열·부분집합처럼 후보가 빠르게 늘면 완전 탐색의 비용이 커집니다.

이제 필요한 것은 문제의 크기 자체를 줄이는 기술입니다.

2장: 완전 탐색

모든 경우를 하나하나 확인했다

한계: 조합적 폭발

조합 문제에서는 후보 수가 빠르게 증가

3장: 축소 정복

쪼개고, 버리고, 핵심만 남긴다

3.1

축소 정복이란 무엇인가?

문제의 크기를 단계적으로 줄여가며 해결 범위를 좁히는 전략을 알아봅니다.

축소 정복(Decrease and Conquer)의 정의

정의

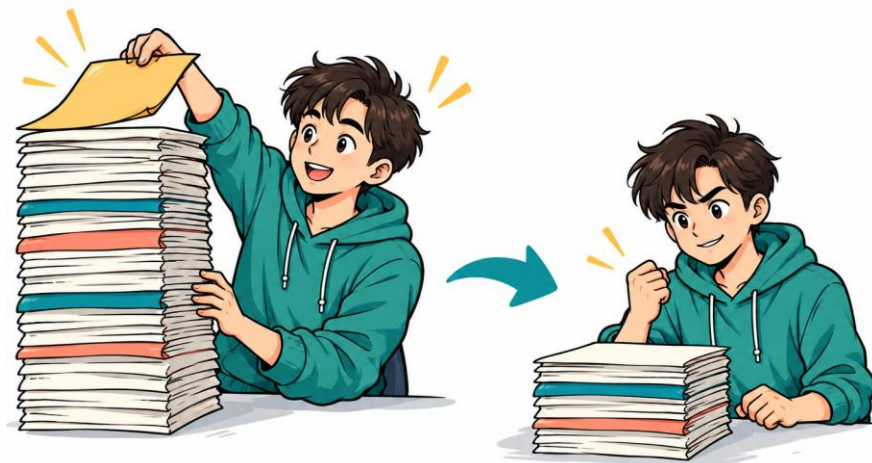
같은 유형의 더 작은 부분 문제 하나를 풀고, 그 답을 이용해 원래 문제를 해결하는 전략입니다.

풀이 과정

1. 문제의 크기를 줄입니다.
2. 작은 문제의 답을 구합니다.
3. 그 답으로 원래 문제의 답을 만듭니다.

예: $5!$ 은 $4!$ 을 구한 뒤 5를 곱하면 됩니다.

더 이상 줄일 필요가 없는 기본 사례에서 멈춥니다.



비유: 100장의 접시와 설거지



한 장씩 씻을 때마다 남은 접시는 99장, 98장으로 줄어든다

한 번에 하나씩 — 매 단계마다 문제의 크기가 정확히 1씩 줄어듭니다 (동일 크기 축소).

문제

접시가 100장 쌓여 있습니다. 먼저 한 장을 씻으면 남은 문제는 99장을 씻는 일입니다.

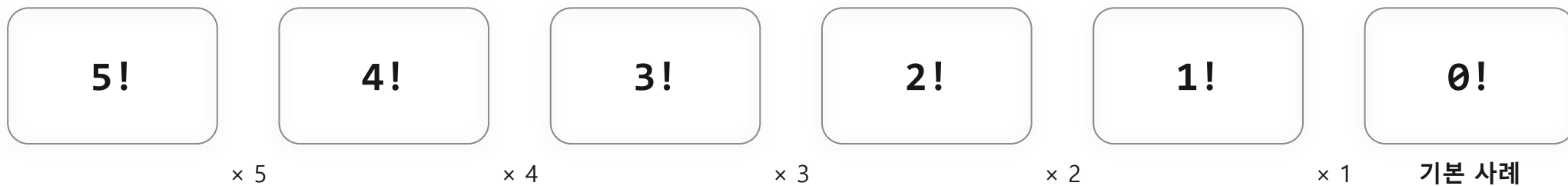
축소 + 정복

한 장을 씻고 나머지 99장에 같은 방법을 적용합니다. 문제의 크기를 하나씩 줄여가며 해결하는 방식입니다.

축소 정복 전략의 대표적인 예: 팩토리얼

$$n! = n \times (n-1)! \quad (\text{단, } 0! = 1)$$

먼저 $(n-1)!$ 을 구하고 n 을 곱합니다. 같은 종류의 문제에서 크기만 하나 줄었습니다.



같은 유형의 더 작은 문제로 바꾸고, 기본 사례 $0! = 1$ 에서 멈춥니다.

3.2

축소의 세 가지 유형

한 번에 문제의 크기를 얼마나 줄이느냐에 따라 축소 정복은 세 갈래로 나뉩니다.

축소 정복의 세 가지 주요 유형

단계 수와 단계당 작업량을 함께 분석합니다.

유형	크기 변화	대표 예	전체 시간 예
동일 크기 축소	$n \rightarrow n - 1$	순차 탐색 / 삽입 정렬	$O(n)$ / 최악 $O(n^2)$
동일 비율 축소	$n \rightarrow n / c$	이진 탐색 / 빠른 거듭제곱	단계당 $O(1)$ 이면 $O(\log n)$
가변 크기 축소	입력 값에 따라 변화	유클리드 최대공약수	나머지 연산 $O(\log b)$ 회

입력 축소 방식만으로 전체 실행 시간을 정할 수는 없습니다.

① 동일 크기 축소 (Decrease by a Constant)

매 단계에서 입력 크기를 일정한 양만큼 줄입니다. 여기서는 하나씩 줄이는 $n - 1$ 형태를 살펴봅니다.

매 단계 크기가 1씩 감소



$$n \rightarrow n - 1$$

단계 수 $\approx n$
단계당 $O(1)$ 이면 $O(n)$

대표 예 팩토리얼 계산 · 순차 탐색 · 삽입 정렬
전체 시간은 단계 수와 각 단계의 작업량을 함께 따져야 합니다.

② 동일 비율 축소 (Decrease by a Constant Factor)

매 단계에서 문제 크기를 일정한 비율로 줄입니다. 절반씩 줄이는 경우에는 n , $n/2$, $n/4$ 순으로 남습니다.



매 단계 크기가 절반으로 감소한다

$$n \rightarrow n / 2$$

단계 수 $\approx \log_2 n$
 단계당 $O(1)$ 이면 $O(\log n)$

대표 예 이진 탐색 · 위조 동전 찾기 · 축소 정복 거듭제곱
 $n = 1,000,000$ 이어도 단계 수는 20번 남짓입니다.

③ 가변 크기 축소 (Variable-Size Decrease)

각 단계에서 입력 크기를 일정하지 않게 줄여 나갑니다. 상황에 따라 감소 폭이 달라져 유연하지만, 분석은 다소 복잡합니다.

유클리드 알고리즘: $\text{gcd}(48, 18)$



감소 폭 30

감소 폭 6

감소 폭 6

종료

$(a, b) \rightarrow (b, a \bmod b)$ — 줄어드는 폭이 매번 다릅니다.

$n \rightarrow ?$

감소 폭이 입력에 의존
분석이 까다롭다

대표 예 유클리드 알고리즘 · 보간 탐색 · 퀵 선택
최댓값을 찾을 때 일부 요소를 한 번에 제거하는 등의 변형된 접근도 여기에 속합니다.

축소 정복 알고리즘의 구현 방식 비교

재귀와 반복은 코드를 실행하는 방식입니다. 하향식과 상향식은 문제를 푸는 방향입니다.

구분	재귀	반복
방식	자기 자신을 호출해 다음 문제를 푼다	반복문으로 상태를 갱신한다
진행 방향	하향식 풀이를 자연스럽게 표현	하향식·상향식 모두 표현 가능
예	팩토리얼·이진 탐색의 재귀 구현	반복 이진 탐색·반복 팩토리얼
특징	부분 문제의 관계가 코드에 드러남	재귀 호출 없이 상태를 갱신
주의	호출 깊이·스택 비용	계산 순서와 저장할 상태 설계

예: 반복 이진 탐색도 큰 탐색 범위를 절반씩 줄이는 하향식 풀이입니다.

재귀·반복과 하향식·상향식은 서로 다른 기준입니다.

하향식 방법과 상향식 방법의 비교

큰 문제에서 시작해 쪼개 내려가느냐, 작은 문제부터 쌓아 올리느냐의 차이입니다.



하향식 Top-down

구하려는 큰 문제에서 출발해 필요한 작은 문제로 내려갑니다.

예: 이진 탐색은 재귀든 반복이든 탐색 범위를 줄입니다.

큰 문제에서 시작

상향식 Bottom-up

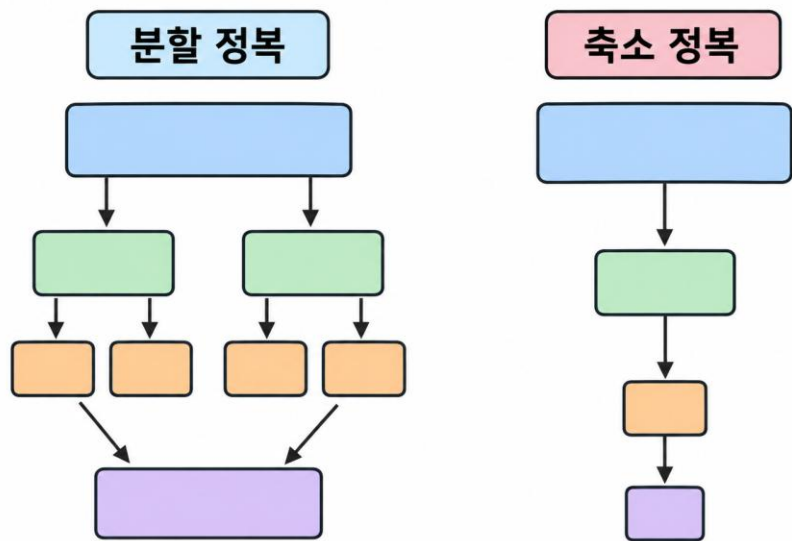
작은 문제의 답부터 계산하고, 그 답으로 더 큰 문제를 풉니다.

예: 반복 팩토리얼은 $1!$ 부터 $n!$ 까지 차례로 계산합니다.

작은 답에서 시작

분할 정복(Divide and Conquer)과의 비교

두 방법 모두 문제를 작게 바꿉니다. 축소 정복은 부분 문제 하나를, 분할 정복은 여러 부분을 풉니다. 표에서 n 은 입력 크기, $T(n)$ 은 전체 기본 연산 횟수이며, 관계식은 작은 문제의 비용과 현재 단계의 비용을 더한 것입니다.



축소 정복은 한 갈래, 분할 정복은 모든 갈래로 내려간다

구분	축소 정복	분할 정복
나누는 방식	하나의 부분 문제로 축소	여러 개(보통 2개)로 분할
푸는 범위	부분 문제 중 하나만 해결	모든 부분 문제를 모두 해결
결합 단계	작은 답으로 결과 구성	여러 부분 답으로 결과 구성
재귀 호출 수	단계당 1회	단계당 2회 이상
대표 알고리즘	이진 탐색, 퀵 선택트	병합 정렬, 퀵 정렬
점화식 예	$T(n) = T(n/2) + 1$	$T(n) = 2T(n/2) + n$

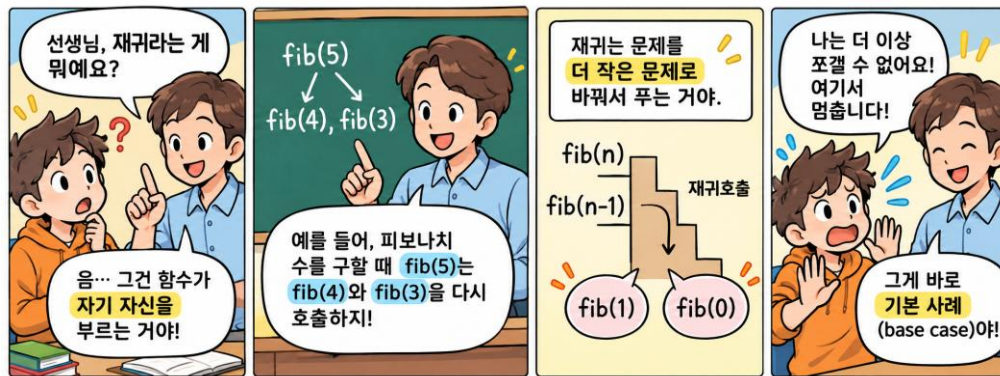
3.3

구현: 재귀와 반복

같은 축소 전략을 재귀 함수와 반복문으로 구현하고, 호출 과정과 저장 공간을 비교합니다.

재귀 (Recursion)

하향식 축소 정복은 문제를 더 작은 하나의 부분 문제로 축소하고, 그 결과를 이용해 원래 문제를 해결합니다.



기본 사례 (base case)

더 이상 쪼갤 수 없는 가장 작은 입력에 대한 즉각적인 해.
여기서 재귀가 멈춘다.

재귀 사례 (recursive case)

문제를 더 작은 동일 문제로 축소해 자기 자신을 호출하고,
그 결과로 해를 구성한다.

재귀 함수의 기본 골격

모든 재귀 함수는 "멈추는 조건"과 "줄여서 다시 부르는 조건" 두 가지로 이루어집니다.

recursion_template.pseudo

```
function Rec(n):  
    if n이 충분히 작다면:  
        return n에 대한 직접적인 해답  
    smaller_problem = n을 더 작은 n'으로 변환  
    sub_result = Rec(smaller_problem)  
    result = sub_result를 이용하여 해답을 구성한다  
    return result
```

기본 사례를 빠뜨리면 무한 재귀 — 호출 스택이 넘쳐 RecursionError가 발생합니다.

팩토리얼 계산 (재귀)

입력 n 은 0 이상의 정수입니다. $n!$ 을 구하는 재귀를 보며, 함수가 호출되고 결과가 돌아오는 순서를 확인합니다.

factorial.py

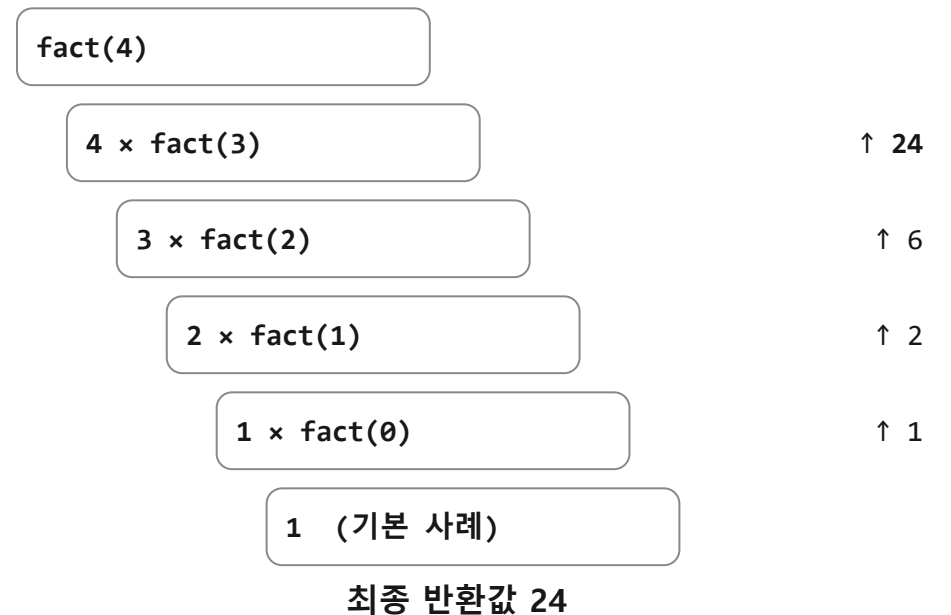
```
def factorial(n):  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

동작 원리

$n = 0$ 에 도달할 때까지 호출을 쌓아 내려가고(하향), 기본 사례에서 값이 확정되면 곱셈을 수행하며 되돌아 올라옵니다(상향).

호출 깊이가 n 이므로 재귀 스택도 n 만큼 쌓입니다.

fact(4) 호출 트레이스



재귀 알고리즘의 복잡도 (팩토리얼)

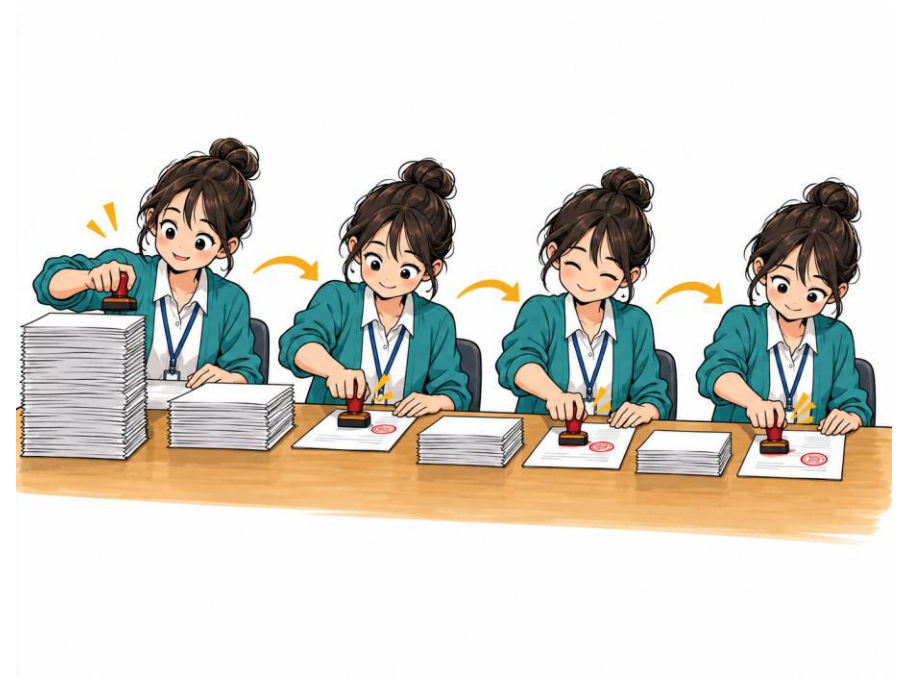
곱셈 횟수를 $M(n)$ 이라고 하겠습니다.
 $n = 0$ 이면 곱셈 없이 1을 반환합니다.

$$M(n) = M(n - 1) + 1, M(0) = 0$$

반복해서 대입하면

$$\begin{aligned} M(n) &= M(n - 2) + 2 \\ &= M(n - 3) + 3 \\ &= M(0) + n = n \end{aligned}$$

곱셈을 상수 시간으로 보면 시간은 $O(n)$ 입니다.
재귀 호출을 저장하는 공간도 $O(n)$ 입니다.



팩토리얼 계산 (반복)

작은 문제를 먼저 해결하고 그것을 바탕으로 큰 문제를 해결합니다. 즉 $1! \rightarrow 2! \rightarrow \dots \rightarrow n!$ 순으로 곱해 나가는 상향식 방법입니다.

factorial_iter.py

```
def factorial_iter(n):  
    result = 1  
    for i in range(1, n + 1):  
        result *= i  
    return result
```

$O(n)$

재귀와 같은 선형 시간이지만
스택을 쓰지 않아 더 안정적

재귀는 사고 표현에, 반복은 실행 성능에 유리 — 축소 정복은 둘 다로 쉽게 구현할 수 있습니다.

피보나치 수열 계산 (재귀)

입력 n 은 0 이상의 정수입니다. $F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$ 입니다. 두 부분 문제를 호출하므로, 모든 재귀가 축소 정복인 것은 아닙니다.

fib.py

```
def fib(n: int):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)
```

$$F(n) = F(n-1) + F(n-2)$$

$$\phi \approx 1.618$$

황금비 (golden ratio)

수열의 앞부분

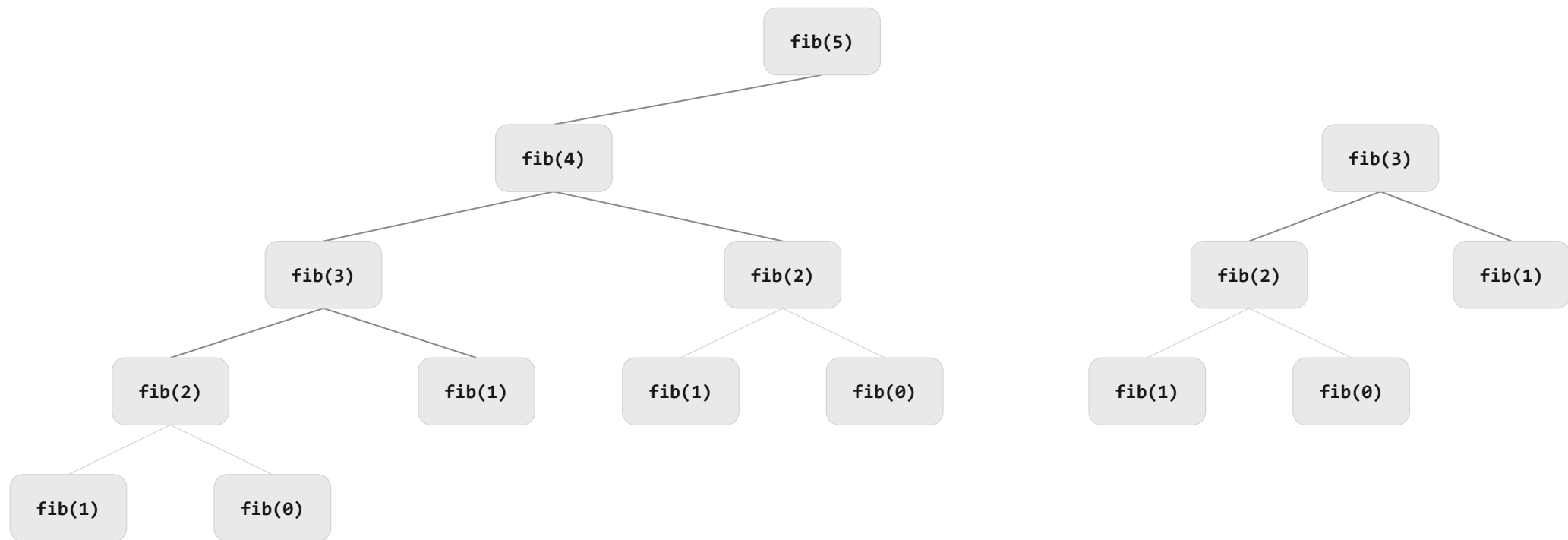
0	1	1	2	3	5	8	13	21	34
F(0)	F(1)	F(2)	F(3)	F(4)	F(5)	F(6)	F(7)	F(8)	F(9)

기본 조건



왜 비효율적인가: 중복 계산되는 부분 문제

같은 값이 여러 번 계산됩니다. fib(5)의 호출 트리에서 fib(2)는 세 번 나타납니다.



fib(2) — 3회 중복 계산

fib(3) — 2회 중복 계산

동일한 부분 문제를 매번 처음부터 다시 계산합니다.

재귀 피보나치의 복잡도: 지수 시간

fib(n)을 실행하며 들어가는 전체 호출·덧셈 등 기본 연산 횟수를 $T(n)$ 이라고 하겠습니다. n 이 0 또는 1이면 바로 반환합니다. 그 외에는 fib($n-1$), fib($n-2$)를 호출하고 두 값을 더합니다. 각 기본 연산은 상수 시간으로 셉니다.

$$T(n) = T(n-1) + T(n-2) + O(1)$$

작은 입력의 비용과 연결하기

$$T(n) > T(n-1) + T(n-2)$$

상수항 버리기

$$T(n) > 2 \times T(n-2)$$

$T(n-1) \geq T(n-2)$ 이용

$$\Omega(2^{(n/2)})$$

$$T(n) > 2 \cdot 2 \cdot \dots \cdot 2 \times T(0)$$

짝수 n 에서 $n/2$ 번 전개

호출 수가 지수적으로 늘어납니다.

$$T(n) > 2^{(n/2)}$$

$T(0) = 1$

n 이 커지면 중복 계산 때문에 실행 시간이 급격히 늘어납니다.

해법 이미 계산한 값을 저장해 두었다가 재사용하면 중복 계산을 피할 수 있습니다.

피보나치 수열 계산 (반복)

값을 몇 개의 변수에 저장해 두고 재사용하면 중복 계산이 사라집니다. 각 항을 한 번씩만 계산하므로 시간 복잡도는 $O(n)$ 입니다.

fib2.py

```
def fib2(n: int):  
    a, b = 0, 1  
    for _ in range(n):  
        a, b = b, a + b  
    return a
```

컴퓨터 성능이 아무리 발전해도
알고리즘의 효율성은 여전히 핵심적인 문제
임을 명확히 보여줍니다.

순진한 재귀: 같은 값을 반복 계산
호출 수가 지수적으로 증가

반복: 각 항을 한 번씩 계산
 n 단계 · 시간 $O(n)$ · 추가 공간 $O(1)$

기본 연산을 상수 시간으로 보는 연산 수 비교

그렇다면 항상 반복이 재귀보다 좋을까?

재귀는 부분 문제의 관계를 코드로 표현하기 좋습니다. 아래 병합 정렬은 두 부분을 모두 푸는 분할 정복 예시입니다.

merge_sort.py (구조 예시, merge는 별도 구현)

```
def merge_sort(A):  
    if len(A) <= 1:  
        return A  
    mid = len(A) // 2  
    left = merge_sort(A[:mid])  
    right = merge_sort(A[mid:])  
    return merge(left, right)
```

표현력

분할·병합 구조를 코드가 그대로 보여준다

성능

반복은 재귀 호출 비용을 줄일 수 있다

선택 기준

구조가 재귀적이면 재귀, 단순 누적이면 반복

문제 구조와 필요한 저장 공간을 함께 보고 재귀와 반복을 선택합니다.

3.4

탐색 알고리즘

순차 탐색.이진 탐색.보간 탐색은 각각 세 가지 축소 유형을 그대로 보여줍니다.

순차 탐색 (동일 크기 축소)

길이 n 인 배열을 인덱스 0부터 검사합니다. 목표값과 처음 일치하는 인덱스를 반환하고 없으면 -1 입니다. 비교 횟수는 배열 원소와 $target$ 의 일치 비교만 셉니다.

linear_search.py

```
def linear_search(arr, target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1
```

target = 7 을 찾는 과정



4번의 비교 끝에 인덱스 3에서 발견
최악의 경우 n 번 모두 비교해야 합니다.

최선

$O(1)$

첫 번째 원소가 정답

평균

$O(n)$

목표가 반드시 있고 각 위치의
확률이 같으면 평균 $(n+1)/2$ 회

최악

$O(n)$

마지막에 있거나 없음

이진 탐색 (동일 비율 축소)

오름차순 배열에서 목표값을 찾습니다. 인덱스는 0부터이며 [left,right]는 양 끝을 포함합니다. $mid = (left + right) // 2$ 에서 //는 정수 몫입니다.

binary_search.py

```
def binary_search(arr, target, left, right):
    if left > right:
        return -1
    mid = (left + right) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search(arr, target, mid+1, right)
    else:
        return binary_search(arr, target, left, mid-1)
```

arr = [1,3,5,7,9,11], target = 7

1	3	5	7	9	11
---	---	---	---	---	----

arr[2]=5 < 7 → 오른쪽

1	3	5	7	9	11
---	---	---	---	---	----

arr[4]=9 > 7 → 왼쪽

1	3	5	7	9	11
---	---	---	---	---	----

arr[3]=7 → 발견!

최초 호출: binary_search(arr, target, 0, len(arr)-1). 찾으면 인덱스, 없으면 -1을 반환합니다.

이진 탐색의 시간 복잡도 분석

한 단계에서 가운데 원소를 확인합니다. 최악의 경우 남은 후보가 절반 이하로 줄어듭니다.



최종 시간 복잡도 = $O(\log n)$

가운데 원소 확인 횟수 $C(n)$

$$C(n) \leq C(\lfloor n/2 \rfloor) + 1$$

$$C(0) = 0, \quad C(1) = 1$$

$$O(\log n)$$

$n = 1,000$

약 10회

$n = 1,000,000$

약 20회

$n = 10^9$

약 30회

보간 탐색 (가변 크기 축소)

오름차순 숫자 배열에서 목표값 x 의 위치를 양 끝 값으로 추정합니다. $low, high$ 는 0부터 세는 후보 양 끝 인덱스이며 $//$ 는 정수 몫입니다. 양 끝 값이 같으면 아래 식을 쓰지 않습니다.

$$pos = low + (x - S[low]) \times (high - low) // (S[high] - S[low])$$

이진 탐색과 무엇이 다른가

이진 탐색은 값과 무관하게 항상 한가운데를 봅니다.
보간 탐색은 값의 크기를 이용해 예상 위치로 곧장 점프하므로, 줄어드는 폭이 매 단계 달라집니다.

시간 복잡도

균등 분포에서 기대 시간 $O(\log \log n)$

최악의 경우 $O(n)$

균등 분포에서 유리하며, 최악에는 $O(n)$ 이 걸립니다.

보간 탐색 — 전체 소스

interpolation_search.py (1/2)

```
def interpolation_search(arr, x):
    low, high = 0, len(arr) - 1
    while low <= high:
        if x < arr[low] or x > arr[high]:
            break
        if arr[low] == arr[high]:
            return low
        offset = (x - arr[low]) * (high - low)
        span = arr[high] - arr[low]
        pos = low + offset // span
```

interpolation_search.py (2/2)

```
        if arr[pos] == x:
            return pos
        if arr[pos] < x:
            low = pos + 1
        else:
            high = pos - 1
    return -1

S = [10, 20, 30, 40, 50, 60, 70, 80, 90]
idx = interpolation_search(S, 70)
print("70의 인덱스:", idx)
```

실행 결과 70의 인덱스: 6 (오름차순 정수 배열)

세 가지 탐색 알고리즘 정리

탐색 방법은 입력 조건과 축소 방식이 다릅니다. 정렬 비용과 데이터 분포도 함께 확인합니다.

알고리즘	축소 유형	전제 조건	평균 복잡도	최악 복잡도
순차 탐색	동일 크기 ($n \rightarrow n-1$)	없음	$O(n)$	$O(n)$
이진 탐색	동일 비율 ($n \rightarrow n/2$)	정렬된 배열	$O(\log n)$	$O(\log n)$
보간 탐색	가변 크기 ($n \rightarrow ?$)	정렬된 수치 배열	균등 분포에서 $O(\log \log n)$	$O(n)$

동일 크기

정렬 없이도 탐색할 수 있다.

동일 비율

정렬된 데이터에서 반복 탐색할 때 유리하다.

가변 크기

분포가 고르면 더 빠를 수 있다.

3.5

유클리드와 거듭제곱

가변 크기 축소의 대표 알고리즘과, 절반씩 줄이는 거듭제곱 계산을 살펴봅니다.

유클리드 알고리즘 (가변 크기 축소)

$\text{gcd}(a,b)$ 는 a 와 b 의 최대공약수를 반환합니다. $a \geq b > 0$ 인 정수에서 (a,b) 를 $(b, a \% b)$ 로 바꾸며, $\%$ 는 나눗셈의 나머지입니다. $b=0$ 이면 a 를 반환합니다. 새 두 번째 값 $a \% b$ 는 이전 b 보다 작아 계속 줄어듭니다.

gcd.py

```
def gcd(a, b):
    if b == 0:
        print(f"b가 0이므로 종료. 최대공약수는 {a}")
        return a
    else:
        return gcd(b, a % b)
```

`gcd(1071, 462)`

`gcd(1071, 462)`

$a \bmod b = 147$

`gcd(462, 147)`

$a \bmod b = 21$

`gcd(147, 21)`

$a \bmod b = 0$

`gcd(21, 0)`

$b = 0 \rightarrow \text{gcd} = 21$

나머지는 나누는 수보다 작습니다. 나머지 연산 횟수는 $O(\log b)$ 입니다.

거듭제곱 계산 — 가장 먼저 떠오르는 방법

정수 C 의 거듭제곱 C^n 을 구합니다. n 은 0 이상의 정수이며 $n=0$ 의 반환값은 1로 정합니다. 1에서 시작해 C 를 n 번 곱하는 반복 구현을 살펴봅시다.

power.py

```
def power(C, n):  
    result = 1  
    for i in range(n):  
        result = result * C  
    return result
```

$O(n)$

$n = 1,000,000$ 이면 곱셈을 백만 번 수행합니다.

핵심 아이디어 — 큰 문제를 작은 문제로 나눈다



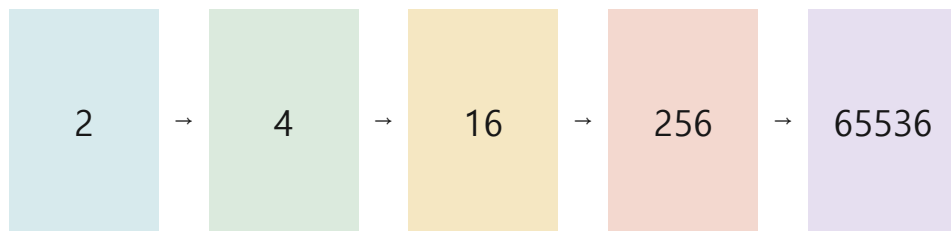
$$2^{100} = (2^{50})^2$$

$$2^{50} = (2^{25})^2$$

$$2^{25} = 2 \times (2^{12})^2$$

축소 정복 거듭제곱의 동작 과정

C^n 은 C 를 n 번 곱한 값입니다. $C^{(n//2)}$ 를 한 번 구해 half에 저장한 뒤 $half \times half$ 를 계산합니다. n 이 홀수이면 C 를 한 번 더 곱합니다.
예: $2^5 = (2^2) \times (2^2) \times 2$.



$$2^{16} = 65536$$

절반으로 내려갔다가 제곱하며 되돌아 올라온다

하강 (축소)

n 은 0 이상의 정수입니다.
//는 정수 몫: $13//2=6$.
 $n \rightarrow n//2 \rightarrow \dots \rightarrow 0$

상승 (결합)

$n=0$ 은 곱셈 없이 1을 반환.
그 외에는 $half \times half$,
홀수면 $\times C$ 를 한 번 더.

결과

곱셈 횟수가 $O(n)$ 에서 $O(\log n)$ 으로 줄어든다

곱셈 횟수 $O(\log n)$
한 단계에서 곱셈 1~2회

축소 정복을 이용한 거듭제곱

fast_power.py

```
def fast_power(C, n):
    if n == 0:
        return 1
    half = fast_power(C, n // 2)
    if n % 2 == 0:
        return half * half
    else:
        return half * half * C
```

2 ^ 13 계산 과정

fast_power(2, 13)	홀수
fast_power(2, 6)	짝수
fast_power(2, 3)	홀수
fast_power(2, 1)	홀수
fast_power(2, 0)	기본 사례

지수 13→6→3→1→0, 호출 5회.
제공 4회+홀수 추가 곱셈 3회=7회.

fast_power(C,n)은 C의 n제곱을 반환합니다. C는 정수, n은 0 이상의 정수이며 $C^0=1$ 로 정합니다. //는 정수 몫, %는 나머지입니다. *를 실행할 때마다 곱셈 1회로 세며, n=0의 반환에는 곱셈이 없습니다.

축소 정복 거듭제곱의 시간 복잡도 분석

n 은 거듭제곱의 지수, $T(n)$ 은 기본 연산의 전체 횟수입니다. `fast_power`는 지수를 절반으로 줄인 호출을 한 번만 하고 그 결과를 저장합니다. 각 단계에서 곱셈은 1~2회이며, 기본 연산 1회를 상수 시간으로 봅니다.

$$T(n) = T(\lfloor n/2 \rfloor) + O(1) \quad (n > 0)$$

$$T(0) = O(1)$$

축소 단계는 약 $\log_2 n$ 번
 각 단계에서 일정한 횟수만큼 곱하므로 곱셈 횟수는 $O(\log n)$ 입니다.
 파이썬의 큰 정수는 자릿수가 늘수록 곱셈 한 번도 더 오래 걸립니다.

$O(\log n)$

반복 횟수 / 축소 단계 (약)

$n = 1,000$	1,000회	10회
$n = 100만$	100만회	20회

실제 실행 시간 측정: 백문이 불여일견

파이썬의 time 모듈로 두 알고리즘의 실행 시간을 직접 측정해 봅시다.

measure.py

```
import time

def measure_time(func, C, n, label):
    start = time.perf_counter()
    func(C, n)
    elapsed = time.perf_counter() - start
    print(label, n, f"{elapsed:.6f}초")

for n in (100000, 1000000):
    measure_time(fast_power, 2, n, "축소 정복")
    measure_time(power, 2, n, "반복문")
```

기존 측정 예시 (환경 미기록)

n	축소 정복	반복문	배수
100,000	0.000187초	0.141415초	약 756배
1,000,000	0.002338초	13.442806초	약 5,749배

표의 시간은 기존 측정 예시입니다.
환경과 정수 자릿수에 따라 달라집니다.

연산 횟수와 실행 시간을 구분하세요. 큰 정수의 곱셈 비용은 자릿수에 따라 증가합니다.

3.6

정렬과 선택 문제

삽입 정렬 · 위조 동전 찾기 · 퀵 선택으로 세 가지 축소 유형을 다시 확인합니다.

삽입 정렬 (동일 크기 축소)

인덱스는 0부터입니다. $i=1$ 부터 $a[i]$ 를 key 에 보관하고 앞의 정렬된 구간에 삽입합니다. 앞 원소 중 key 보다 큰 값만 오른쪽으로 한 칸씩 이동합니다.

insertion_sort.py

```
def insertion_sort(A):
    for i in range(1, len(A)):
        key = A[i]
        j = i - 1
        while j >= 0 and A[j] > key:
            A[j + 1] = A[j]
            j -= 1
        A[j + 1] = key
```

정렬된 구간이 1씩 자란다

5	2	4	6	1	정렬 1개
2	5	4	6	1	정렬 2개
2	4	5	6	1	정렬 3개
2	4	5	6	1	정렬 4개
1	2	4	5	6	완료

$n-1$ 개를 먼저 정렬하고 1개를 끼워 넣는다 — 문제 크기를 한 번에 1씩 줄이는 전형적인 동일 크기 축소입니다.

삽입 정렬의 시간 복잡도

최선 — 이미 오름차순인 입력
 각 단계의 값 비교가 한 번이면 끝납니다. $n-1$ 단계이므로 $\Theta(n)$ 입니다.

최악 — 서로 다른 값의 내림차순 입력
 앞 원소를 한 칸 미는 대입만 이동으로 셉니다.
 key 저장·마지막 삽입은 이동 수에서 제외합니다.
 $1+2+\dots+(n-1)=n(n-1)/2 \rightarrow \Theta(n^2)$

입력 순열이 같은 확률일 때 평균 시간 $\Theta(n^2)$.



위조 동전 문제 (동일 비율 축소)

n 개 동전 중 가짜는 정확히 하나이며 진짜보다 가볍습니다. 진짜 동전은 모두 같은 무게입니다. 양팔저울은 양쪽 더미의 무게가 같은지, 어느 쪽이 가벼운지를 알려 줍니다. 두 더미와 세 그룹으로 줄이는 방법을 비교합니다.

동전 9개를 비교한다면

왼쪽 4개와 오른쪽 4개를 저울에 올리고, 1개는 따로 둡니다.
무게가 다르면 가벼운 4개만 남기고, 같으면 따로 둔 1개가 답입니다.

무게가 같다

홀수일 때 따로 뺀 동전이 가짜

무게가 다르다

더 가벼운 더미에 가짜가 있다

반복

동전이 하나 남을 때까지 계속

탐색 범위가 매번 절반으로 — 동일 비율 축소 ($n \rightarrow n/2$)입니다.

위조 동전 찾기 (2개로 나누기) — 전체 소스

find_fake_coin.py

```
def find_fake_coin(coins, start=0):
    n = len(coins)
    if n == 1:
        return start
    mid = n // 2
    first = coins[:mid]
    second = coins[mid:2 * mid]
    diff = sum(first) - sum(second)
    if diff == 0:
        return start + 2 * mid
    if diff < 0:
        return find_fake_coin(first, start)
    return find_fake_coin(second, start + mid)
```

run.py

```
coins = [1,1,1,1,0.9,1,1,1,1]
index = find_fake_coin(coins)
print(f"가짜 동전의 위치: {index}")
```

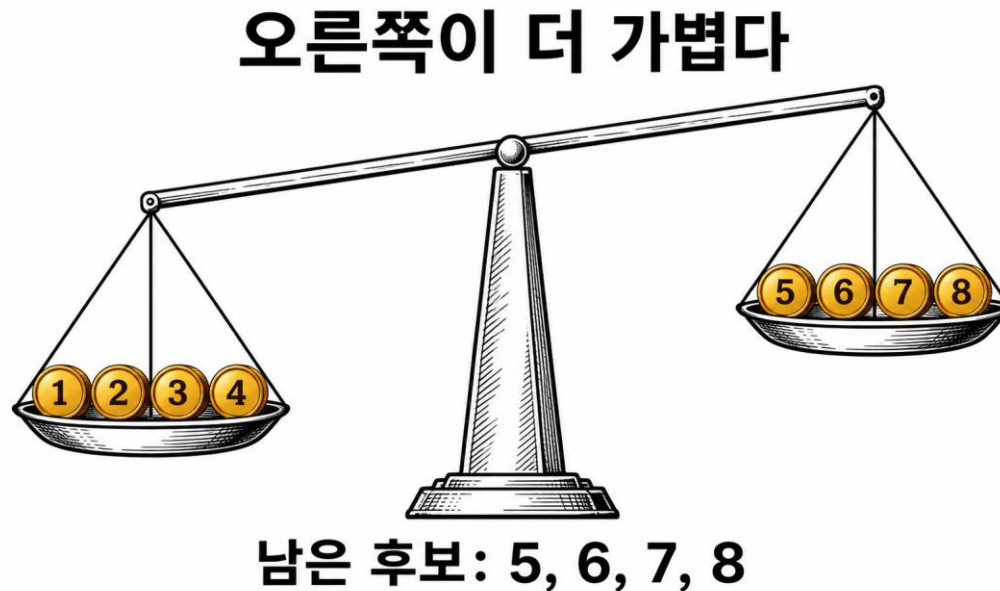
가짜 동전의 위치: 4
(인덱스는 0부터 시작)

복잡도

저울질 횟수 $O(\log n)$
합산·복사를 포함한 시간 $O(n)$

위조 동전 찾기 (2개로 나누기)

n 개의 동전을 두 더미로 나눕니다. 홀수라면 한 개를 따로 빼 둡니다. 두 더미의 무게가 같으면 빼 둔 동전이 가짜이고, 아니면 가벼운 더미로 범위를 좁힙니다.



두 더미로 나누어 저울질 — 한 번에 후보의 절반이 사라진다

위조 동전 찾기 (3개로 나누기)

find_fake_coin3.py

```
def find_fake_coin3(coins, start=0):
    n = len(coins)
    if n == 1:
        return start
    size = (n + 1) // 3
    first = coins[:size]
    second = coins[size:2 * size]
    diff = sum(first) - sum(second)
    if diff < 0:
        return find_fake_coin3(first, start)
    if diff > 0:
        return find_fake_coin3(second, start + size)
    return find_fake_coin3(coins[2 * size:],
                           start + 2 * size)
```

세 그룹의 크기를 최대한 고르게 나눕니다. 같은 개수인 두 그룹을 비교하고, 결과에 따라 한 그룹만 남깁니다. 예: 8개는 3:3:2개로 나눕니다.

```
coins = [10] * 8
coins[6] = 9
print(find_fake_coin3(coins))
# 출력: 6
```

저울질 횟수

최악 $\lceil \log_3 n \rceil$ 회

8개도 최대 2회

합산·복사를 포함한 시간 $O(n)$

sum의 차이로 저울을 흉내 냅니다.

k번째 작은 수 찾기 (선택 문제)

n개의 숫자 중 k번째로 작은 수를 찾는 문제입니다. 중앙값이나 상위 10% 경계값을 구하는 상황이 여기에 해당합니다. 정렬 후 k-1번 인덱스를 읽으면 되지만, 일반적인 비교 정렬은 $O(n \log n)$ 시간이 듭니다.

정렬을 하지 않고도 이보다 빠르게 답을 찾을 수 있을까?

피벗 값 v를 기준으로 작은 값·같은 값·큰 값으로 나눕니다.



$$k \leq L$$

작은 값 그룹에서 k번째를 찾는다.

$$L < k \leq L + E$$

피벗 값 v가 정답이다.

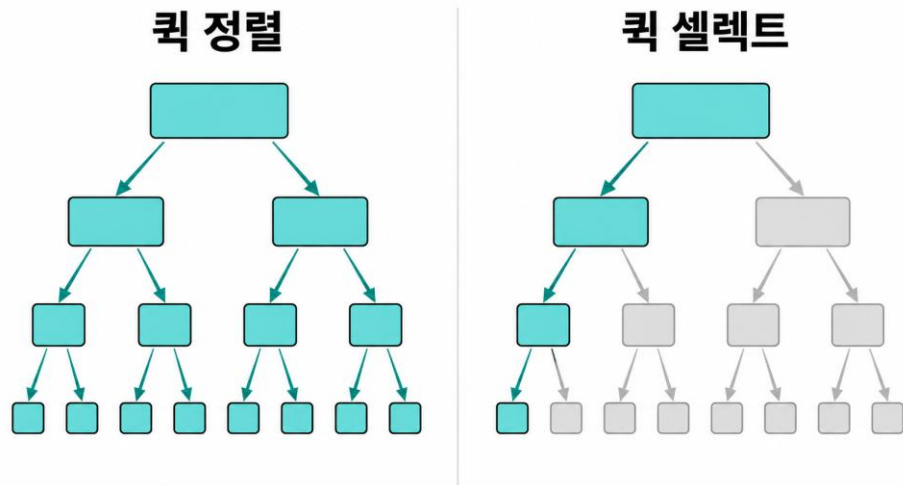
$$k > L + E$$

큰 값 그룹에서 $k - L - E$ 번째를 찾는다.

L: 작은 값의 개수 E: 같은 값의 개수 k: 1부터 세는 순위

퀵 정렬 vs 퀵 선택

퀵 선택은 k번째로 작은 값이 있는 한쪽만 풀고, 퀵 정렬은 양쪽을 모두 정렬합니다. n 은 현재 원소 수, $T(n)$ 은 전체 기본 연산 횟수입니다. 아래 $+n$ 은 현재 원소들을 기준값과 비교하여 분류하는 작업입니다.



퀵 정렬 (분할 정복)

두 부분 배열을 모두 정렬합니다. 아래 식은 두 부분의 크기가 균형 잡힌 경우입니다.

$$T(n) = 2T(n/2) + n$$

퀵 선택 (축소 정복)

정답이 있는 한쪽만 풉니다. 아래 식은 남은 부분의 크기가 약 절반인 경우입니다.

$$T(n) = T(n/2) + n$$

분할 비율은 입력에 따라 달라집니다. 두 알고리즘 모두 최악의 시간은 $O(n^2)$ 입니다.

퀵 셀렉트 — 전체 소스

quick_select.py (1/2)

```
from random import choice

def quick_select(arr, k):
    if not 1 <= k <= len(arr):
        raise ValueError("k out of range")
    pivot = choice(arr)
    small, mid, big = [], [], []
    for x in arr:
        if x < pivot:
            small.append(x)
```

run.py

```
nums = [10, 4, 5, 8, 6, 11, 26]
print("3번째 작은 수:", quick_select(nums, 3))
```

quick_select.py (2/2)

```
        elif x > pivot:
            big.append(x)
        else:
            mid.append(x)
    L, E = len(small), len(mid)
    if k <= L:
        return quick_select(small, k)
    if k <= L + E:
        return pivot
    return quick_select(big, k - L - E)
```

실행 결과 3번째 작은 수: 6
무작위 피벗의 기대 시간 $O(n)$
최악의 시간 $O(n^2)$

퀵 셀렉트의 시간 복잡도 분석

n 은 현재 후보의 원소 수, $T(n)$ 은 전체 기본 연산 횟수입니다.
기준값보다 작은지.같은지.큰지 한 번 분류하는 작업은 $O(n)$ 입니다. 그다음 정답이 있는 한 그룹만 처리합니다.

매번 후보가 절반 남으면

$$T(n) = T(n/2) + O(n)$$

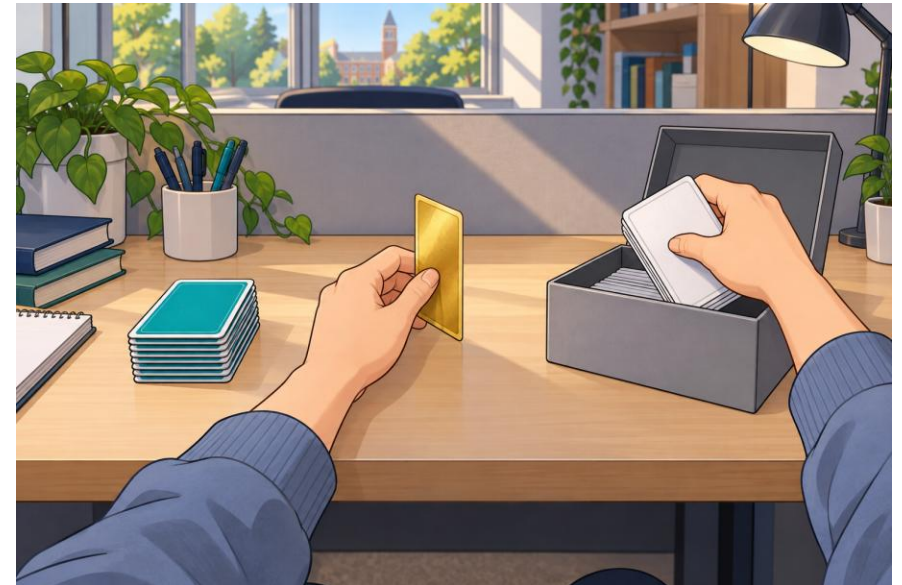
$$\text{단계별 작업량 } n + n/2 + n/4 + \dots < 2n$$

계속 한 개만 줄면

$$T(n) = T(n-1) + O(n)$$

$$\text{단계별 작업량 } n + (n-1) + \dots \rightarrow \text{최악 } O(n^2)$$

무작위로 기준값을 고르면, 선택 결과들을 확률에 따라 평균 낸
기대 시간은 $O(n)$ 입니다.



3.7

Coding Test

별 무늬는 작은 패턴 하나를 재사용합니다. 계단과 로봇은 여러 선택지를 비교하며 동적 계획법으로 연결합니다.

Coding Test 3.1 — 별 모양 찍기

N은 1, 3, 9처럼 3의 거듭제곱입니다. $N = 1$ 이면 별 하나입니다. $N > 1$ 이면 아래 규칙으로 정사각형 무늬를 만듭니다.

- 1 전체 패턴을 3등분하여 총 9개의 작은 영역으로 나눈다. (3×3 격자)
- 2 가운데 영역은 공백으로 비워 두고, 나머지 8개 영역은 크기 $N/3$ 의 패턴으로 채운다.
- 3 크기 $N/3$ 의 패턴을 한 번 구해 8개 위치에 재사용한다.



작은 패턴 하나를 구해
가운데를 뺀 8칸에 재사용

크기를 입력하시오: 9

```

*****
*  *  *  *
*****
***      ***
*  *      *  *
***      ***
*****
*  *  *  *
*****
  
```

Coding Test 3.1 — 전체 소스

make_star.py

```
def make_star(n):
    if n == 1:
        return ['*']
    prev = make_star(n // 3)
    result = []
    for i in range(3):
        for line in prev:
            if i == 1:
                result.append(line + ' ' * (n // 3) + line)
            else:
                result.append(line * 3)
    return result
N = int(input("크기를 입력하시오: "))
for line in make_star(N):
    print(line)
```

기본 사례

$n == 1$ 이면 별 하나짜리 패턴 ['*']

재귀 사례

크기 $n/3$ 의 패턴을 먼저 얻는다

결합

가운데 줄만 공백을 끼워 넣고 나머지는 3번 반복

**재귀 호출은 한 갈래
출력할 문자는 N^2 개
전체 시간 $O(N^2)$**

Coding Test 3.2 — 계단 오르기

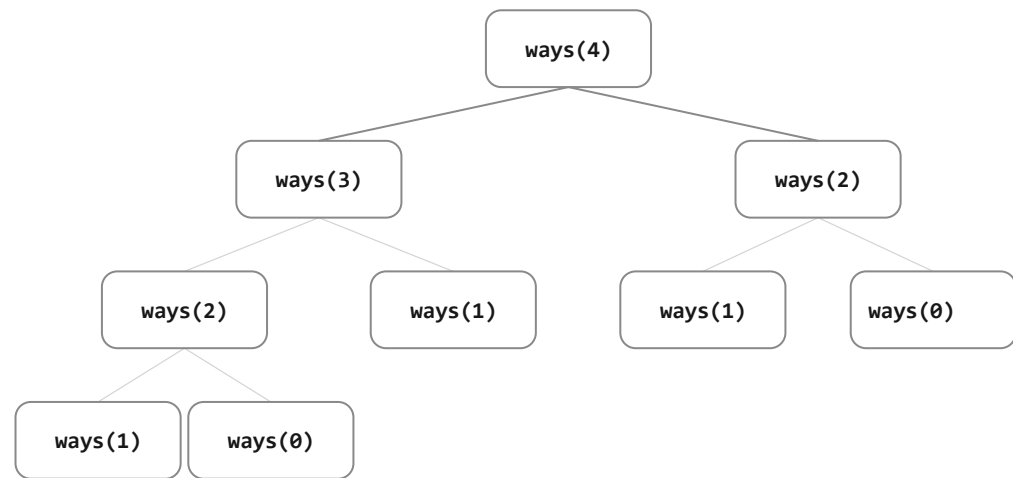
N은 0 이상의 정수입니다. 한 번에 1칸 또는 2칸씩 올라 정확히 N칸에 도착하는 이동 순서의 수를 구하세요. N=0은 아무 이동도 하지 않는 한 가지로 셉니다.

작은 계단 수의 답과 연결하기

- 첫 번째 계단을 올라가는 법: 1가지
- 두 번째 계단을 올라가는 법: 2가지 (1+1 또는 2)
- k층에 올라가는 법 = (k-1)층에서 1칸 + (k-2)층에서 2칸

ways(k)는 정확히 k칸에 도착하는 이동 순서의 수입니다.
 $k \geq 2$ 이면 $ways(k) = ways(k-1) + ways(k-2)$ 입니다.

ways(4) 재귀 트리 — 답은 5



ways(0) = ways(1) = 1
다섯 말단의 값을 더하면 5

Coding Test 3.2 — 전체 소스

ways.py

```
def ways(n):  
    if n < 0:  
        return 0  
    if n <= 1:  
        return 1  
    return ways(n - 1) + ways(n - 2)  
  
N = int(input("계단 개수: "))  
print(ways(N))
```

그림처럼 $n = 0$ 과 $n = 1$ 에서 멈춥니다.
큰 입력에서는 같은 값을 여러 번 계산합니다.

계단 개수: 4
5

관찰

두 부분을 모두 호출하는 재귀입니다.
반복·메모이제이션으로 중복 계산을 줄
일 수 있습니다.

$O(2^n) \rightarrow O(n)$

Coding Test 3.3 — 에너지 절약 로봇

X에서 1까지 가는 최소 횟수를 구합니다. 여러 선택지를 비교하므로 한 갈래 축소 정복과 다릅니다. 아래 코드는 작은 답부터 저장하는 동적 계획법입니다.

X - 1

언제나 가능

X ÷ 2

X가 2로 나누어떨어질 때

X ÷ 3

X가 3으로 나누어떨어질 때

min_energy.py (상향식 동적 계획법)

```
def min_energy(x):
    dp = [0] * (x + 1)
    for v in range(2, x + 1):
        dp[v] = dp[v - 1] + 1
        if v % 2 == 0:
            dp[v] = min(dp[v], dp[v // 2] + 1)
        if v % 3 == 0:
            dp[v] = min(dp[v], dp[v // 3] + 1)
    return dp[x]
print(min_energy(12)) # 3
```

dp[v]: v에서 1까지의 최소 횟수
입력 조건: 양의 정수 x
시간 O(X), 공간 O(X)

12 → 4 → 2 → 1
3번이면 도착합니다.

이미 구한 답은 dp에서 읽습니다.

3장 정리

작은 부분 문제 하나의 답으로 원래 문제를 푹니다. 전체 시간은 축소 단계와 각 단계의 작업량을 함께 따집니다.

축소 유형	크기 변화	대표 알고리즘	전형적 복잡도
동일 크기 축소	$n \rightarrow n - 1$	팩토리얼, 순차 탐색, 삽입 정렬	$O(n) \sim O(n^2)$
동일 비율 축소	$n \rightarrow n / c$	이진 탐색, 위조 동전, 빠른 거듭제곱	$O(\log n)$ 단계 연산 비용 별도
가변 크기 축소	$n \rightarrow ?$	유클리드, 보간 탐색, 퀵 선택	알고리즘별 상이 퀵 선택 최악 $O(n^2)$

재귀 vs 반복

코드를 실행하는 방식
하향식·상향식과 구분합니다.

시간 분석

줄어드는 크기와
단계당 작업을 함께 봅니다.

분할 정복과의 차이

축소 정복은 부분 문제 하나,
분할 정복은 여러 부분을 푹니다.

Q & A

다음 장에서는 여러 부분 문제를 모두 푸는 분할 정복을 배웁니다.